

BC Experiment-7

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

MetaMask is a cryptocurrency wallet and gateway to blockchain applications. It allows users to store, send, and receive Ether or tokens, and connect to decentralized applications (DApps) directly from a web browser. MetaMask acts as a bridge between a user's browser and the Ethereum blockchain (or other compatible networks).

- **Features:**
 - Easy management of multiple blockchain accounts
 - Secure private key storage
 - Integration with test networks like Sepolia, Linea, or RSK
 - Interaction with DApps and smart contracts via Remix IDE
 - **Testnet:**

A Testnet is a blockchain network used for testing and development without using real cryptocurrency. It mimics the main Ethereum network but uses test Ether (fake ETH). Common testnets include Sepolia Testnet, Linea Testnet, and RSK Testnet.
 - **Advantages:**
 - No real money loss during testing
 - Safe environment for developers
 - Helps test smart contracts and transactions before mainnet deployment
- Steps to Connect MetaMask with Remix IDE for Performing Transactions:
- **Set Up MetaMask:**
 - Install the MetaMask extension on your browser.
 - Create or import a wallet account using a recovery phrase.
 - **Add a Test Network:**
 - Open MetaMask → Settings → Networks → Add Network.
 - Enter details for the desired Testnet (e.g., Sepolia or RSK).
 - Example RPC URL: `https://sepolia.infura.io/v3/<your_project_id>`
 - Fund the Wallet:
 - Use a faucet to receive free test Ether (e.g., 0.5 ETH/day).
 - Check the balance in MetaMask.

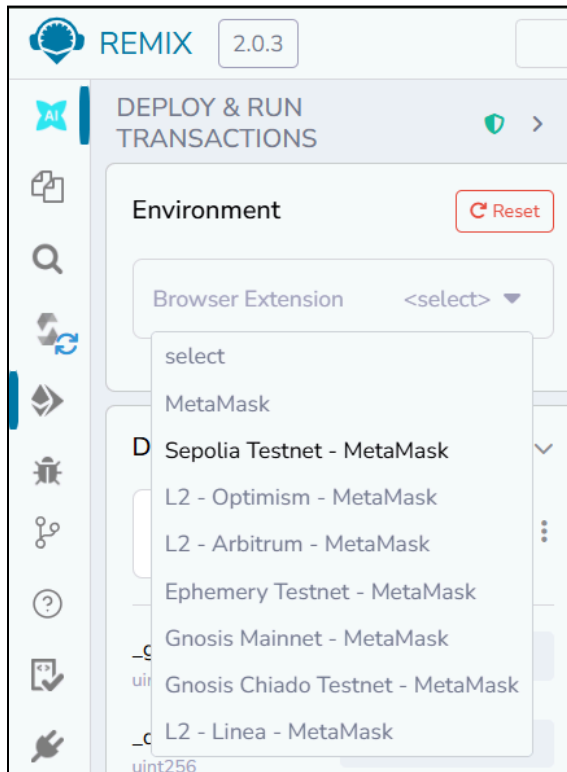
Connect MetaMask with Remix IDE:

Open Remix IDE.

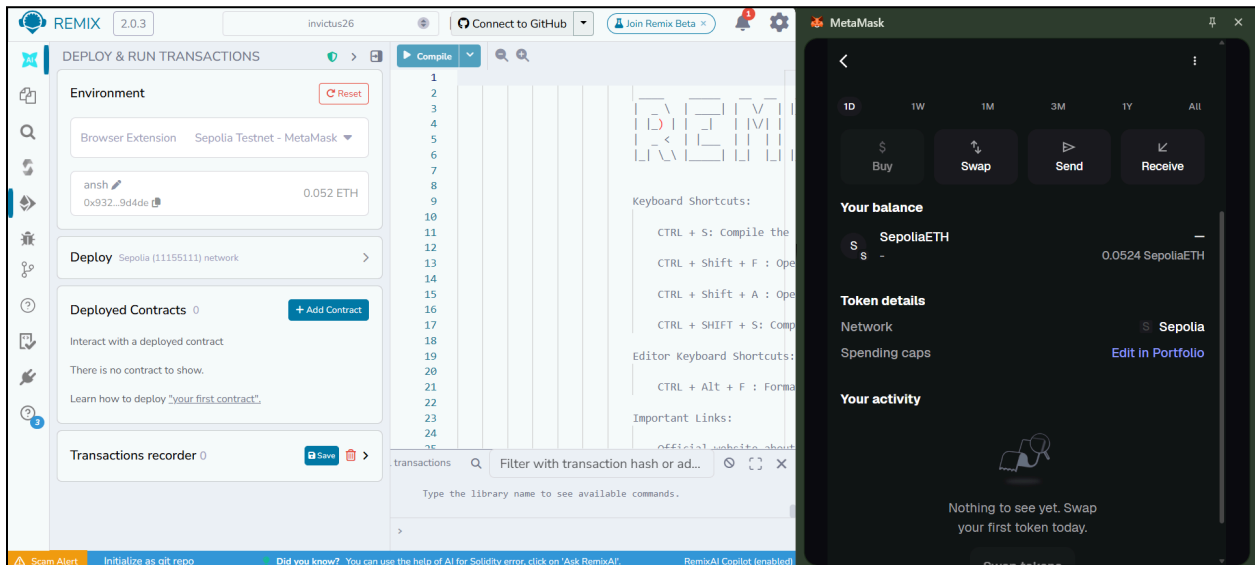
In the Deploy & Run Transactions tab, choose Injected Web3 as the environment.

Approve connection in MetaMask.

Step 1: Select metamask sepolia testnet in environment



Initial setup:



Step 2: Create a contract for a crowdfunding platform

Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;

contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;

    mapping(address => uint) public contributions;

    constructor(uint _goal, uint _duration) {
        owner = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _duration;
    }

    function contribute() public payable {
        require(block.timestamp < deadline, "Deadline passed");
        require(msg.value > 0, "Send some ETH");

        contributions[msg.sender] += msg.value;
        totalFunds += msg.value;
    }

    function withdrawFunds() public {
        require(msg.sender == owner, "Only owner");
        require(totalFunds >= goal, "Goal not reached");

        (bool success, ) = payable(owner).call{value: totalFunds}("");
        require(success, "Transfer failed");
    }

    function refund() public {
        require(block.timestamp > deadline, "Not ended");
        require(totalFunds < goal, "Goal was reached");

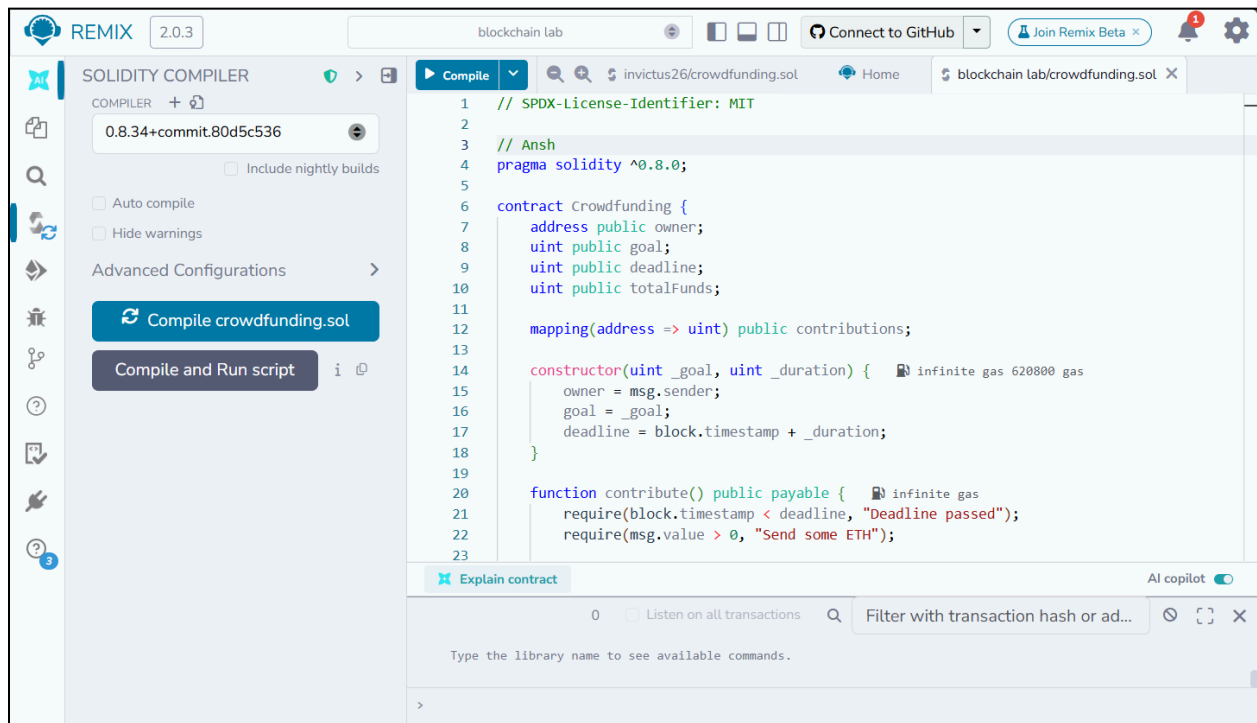
        uint amount = contributions[msg.sender];
        require(amount > 0, "No contribution");
    }
}
```

```
contributions[msg.sender] = 0;
```

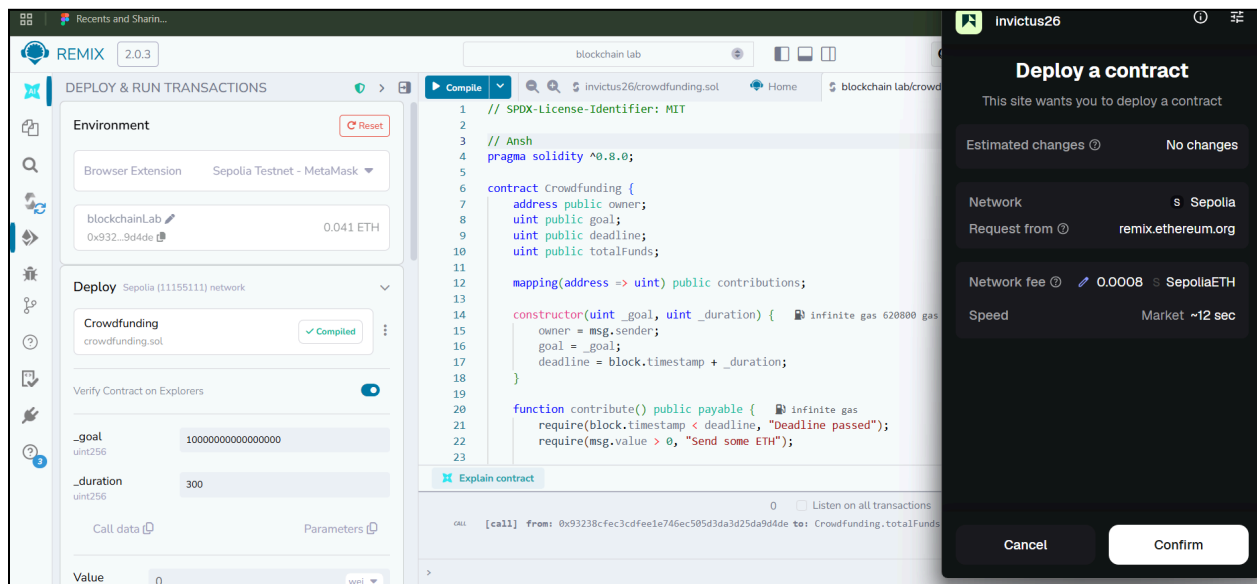
```
(bool success, ) = payable(msg.sender).call{value: amount}("");  
require(success, "Refund failed");  
}
```

```
function getBalance() public view returns (uint) {  
    return address(this).balance;  
}  
}
```

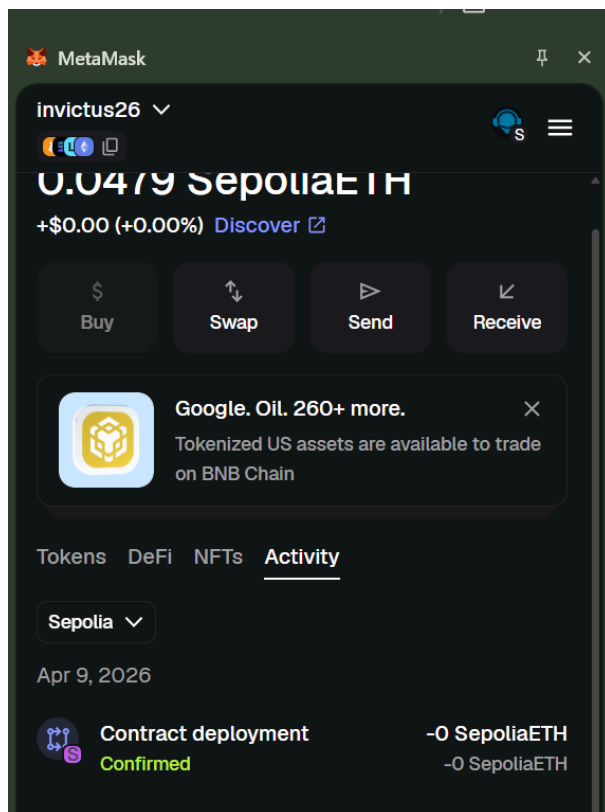
Step 3: save it as **crowdfunding.sol** and compile it



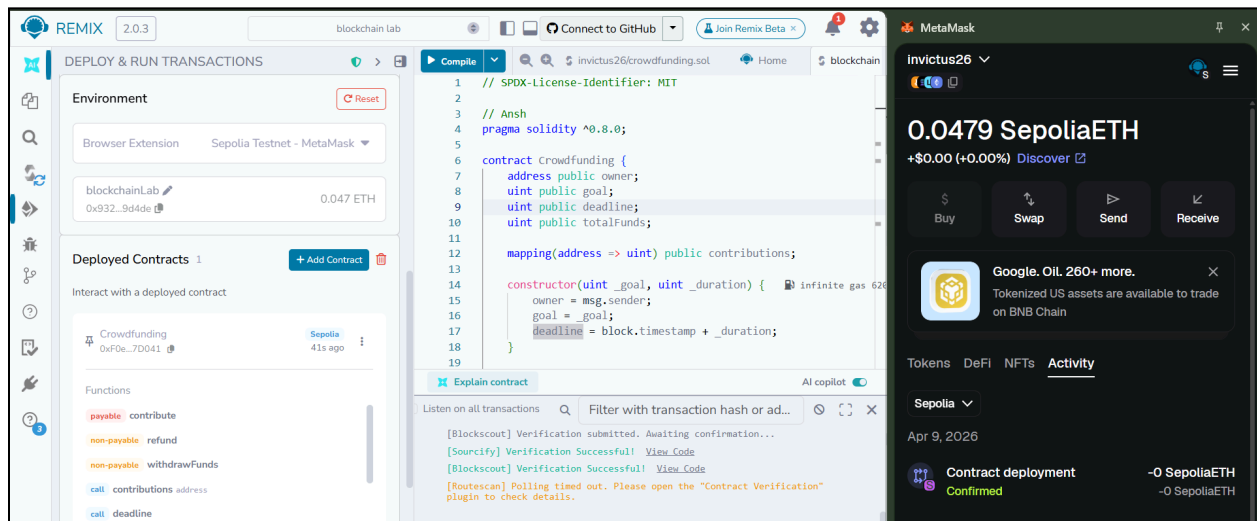
Step 4: deploy the contract with an initial goal and time limit. Confirm the transaction from metamask



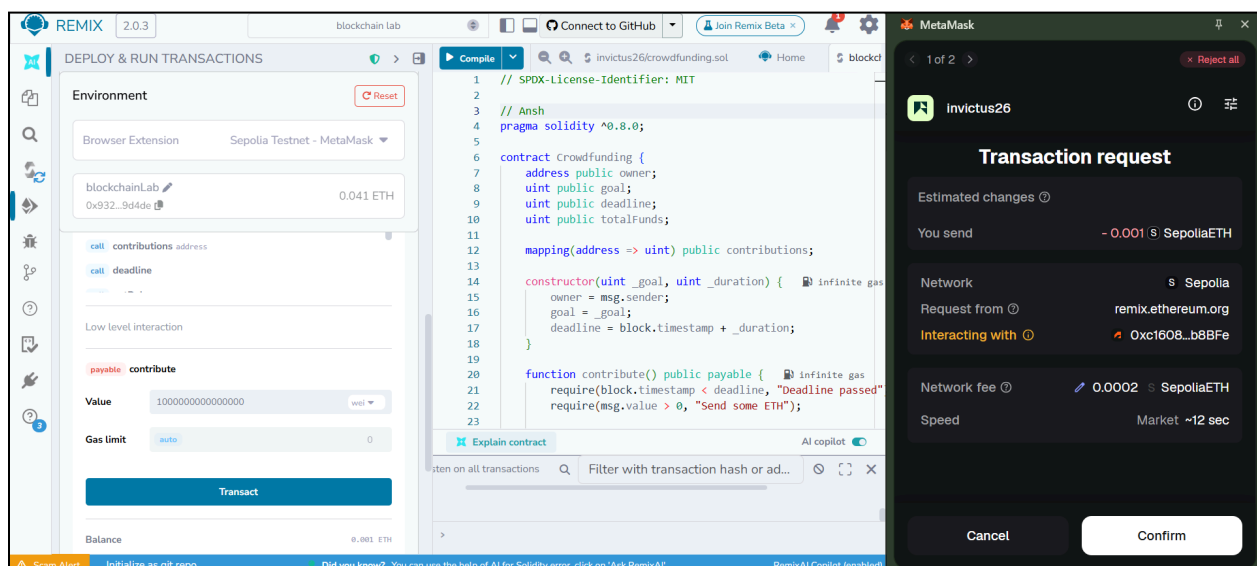
After deployment you can see: transaction recorded in metamask

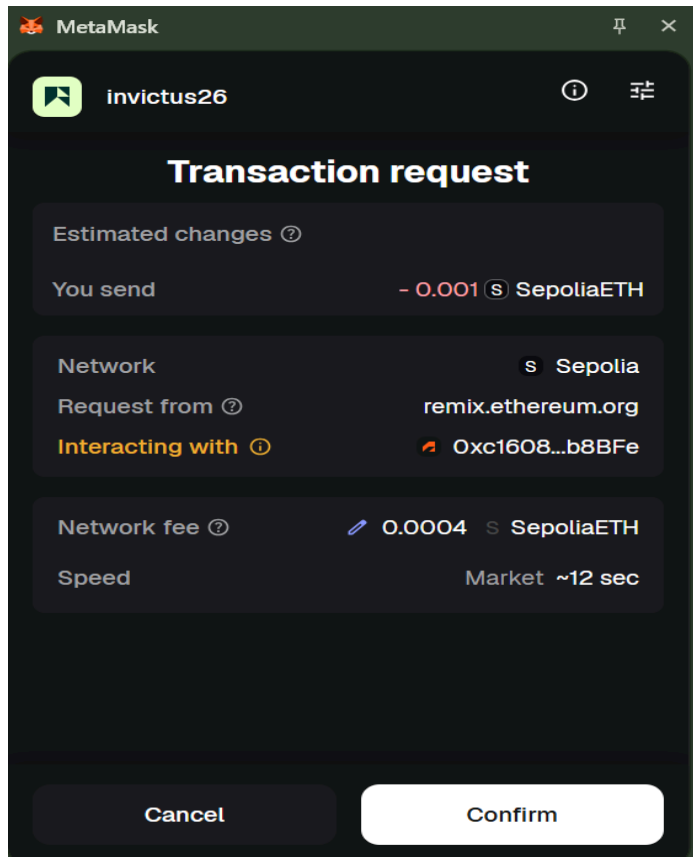


Step 5: After deployment in remix ide, we can see the deployed contract and its functions

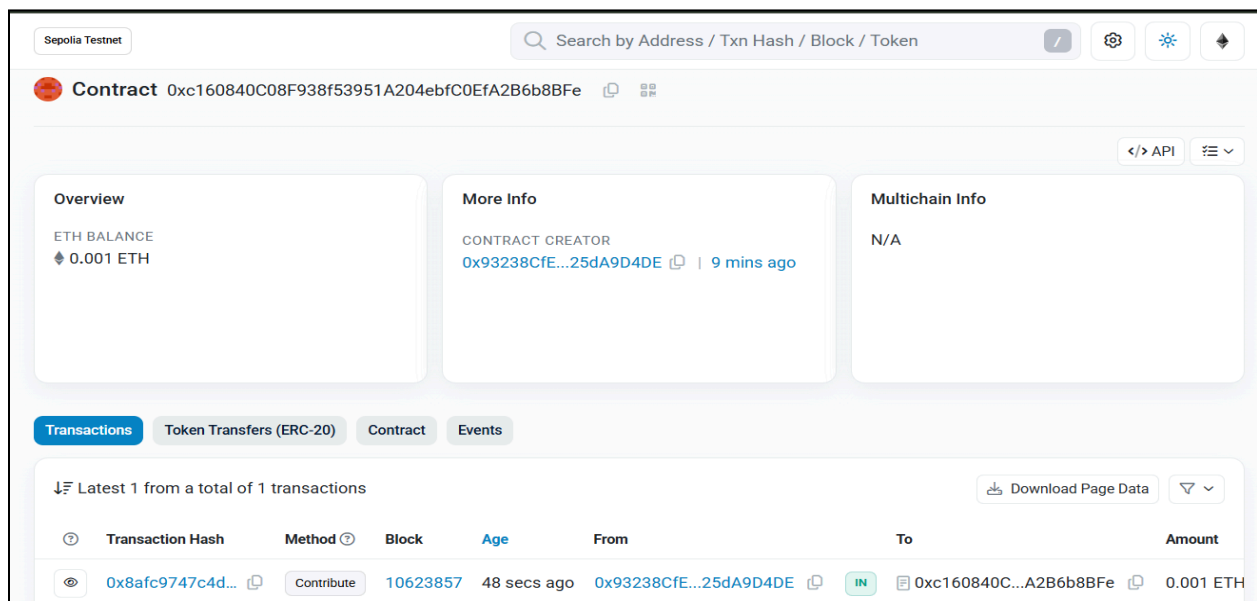


Step 6: Add a contribution of 0.01 ETH = 10000000000000000 wei. Transaction request appears in metamask. Click on confirm

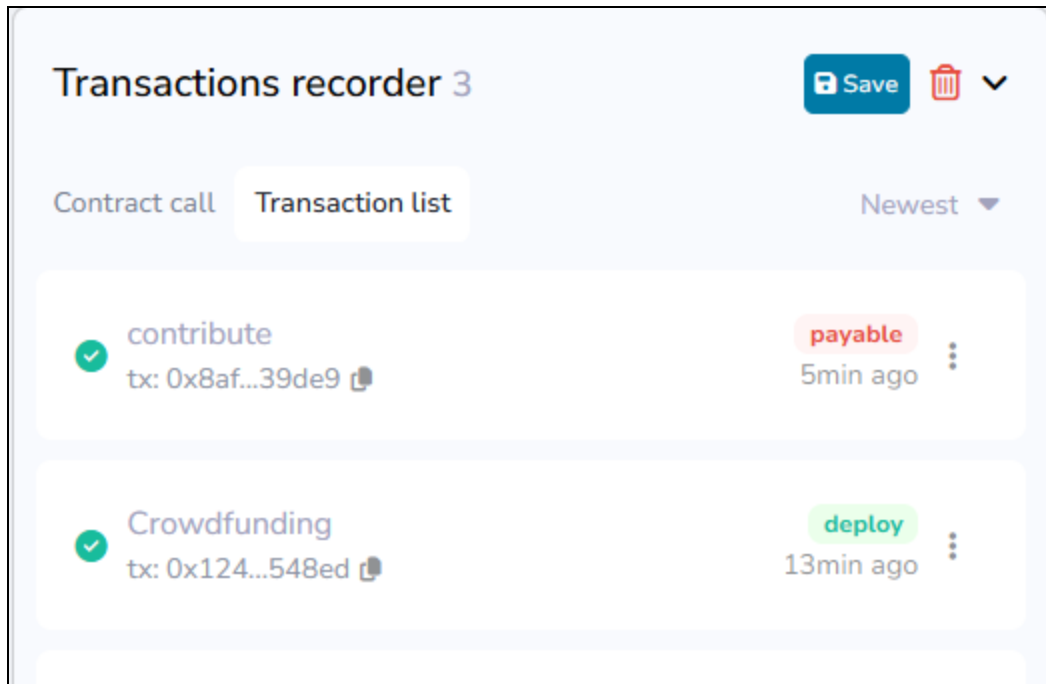




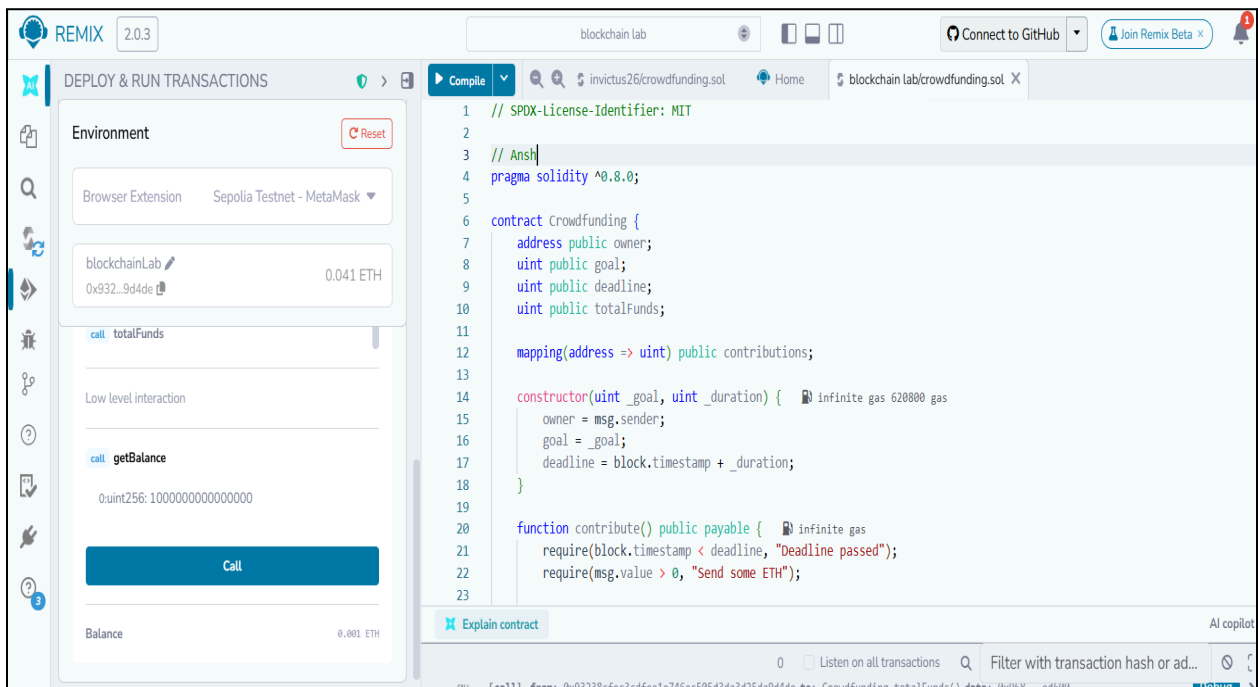
Step 7: copy paste the contracts address in <https://sepolia.etherscan.io/> to see all transactions:



Also can see the transactions under all transactions in remix IDE



Step 8: use getBalance function to check the total funds received



Output:

call

getBalance

0:uint256: 10000000000000000

Call

Balance

0.001 ETH

Step 9: Paste wallet address from metamask into contributions to see all transactions by the particular use

call

contributions

address

0x93238CfEc3cdfEE1E746eC505d3DA3D25dA9D4DE

0:uint256: 10000000000000000

Call

Step 10: use totalFunds function to see totalFunds recieved

call

totalFunds

0:uint256: 10000000000000000

Call

Conclusion:

Using MetaMask with Remix IDE provides a practical way to test and deploy smart contracts on test networks. It helps developers understand real blockchain interactions, transaction confirmations, and wallet integrations in a secure, no-risk environment.